

Day 2 — Timers, Counters & Troubleshooting: Full Instructor Teaching Material (Technical Edition)

Total session time: 210 minutes (3.5 hrs). Each section is sized to its time budget with talking points, board work, worked demos, ladder/ST code, register-level detail, and explicit framing lines.

Addressing convention used throughout this document (adjust to whatever platform the lab actually runs — Allen-Bradley/RSLogix, Siemens TIA/S7, or CODESYS/IEC 61131-3 generic):

- IEC generic: `%I0.0` (input bit), `%Q0.0` (output bit), `%M0.0` (internal memory bit), `%MW10` (memory word)
 - Allen-Bradley style: tag-based, e.g. `Start_PB`, `Motor_Run` (no raw address needed once tags are used, per Day 1)
 - Siemens style: `I0.0`, `Q0.0`, `M0.0`, `MW10`, `DB1.DBX0.0` for data-block bits
-

1. Timers: TON, TOF — 30 min

1a. TON (Timer On-Delay) — 12 min

Function block signature (IEC 61131-3 standard form):

```
TON_Instance(IN := StartCondition, PT := T#5s);  
Q  := TON_Instance.Q;    // BOOL, true when ET >= PT  
ET := TON_Instance.ET;    // TIME, elapsed time, counts up while IN = TRUE
```

Parameters, with actual data types:

- `IN` — BOOL, the timer's enable/input condition
- `PT` — TIME (IEC) or DINT milliseconds (Allen-Bradley uses a DINT `.PRE` in raw milliseconds, e.g. `5000` for 5 seconds)
- `ET` — TIME or DINT `.ACC` (accumulated value, Allen-Bradley), counts up in milliseconds while IN is true
- `Q` — BOOL, true once $ET \geq PT$, false otherwise

Internal state machine, stated precisely (this is what's actually happening under the hood each scan):

```

IF IN = TRUE THEN
    IF ET < PT THEN
        ET := ET + (time since last scan)    // typically the platform's scan-
time delta
    END_IF
    IF ET >= PT THEN
        Q := TRUE
    END_IF
ELSE
    ET := 0
    Q := FALSE
END_IF

```

Note explicitly: **ET** accumulates by *actual elapsed wall-clock time between scans*, not by a fixed "one tick per scan" — this is why timer accuracy is largely independent of scan-time variation (a platform typically uses a hardware clock/counter for this, not scan-count), whereas something like a naively-implemented counter is genuinely scan-dependent (this distinction becomes directly relevant in Block 2c).

Timing diagram (draw this exact shape on the board):



Ladder rendering (Allen-Bradley style, one rung):

```

|--[ Start_PB ]----- ( TON )--|
                          Timer: MotorDelay
                          Preset: 5000 (ms)
                          Accum: 0

```

Critical behavior, stated as a rule to memorize: ET resets to 0 the scan *after* IN goes false — it is not held or paused. This is non-negotiable behavior for a standard (non-retentive) TON and is what distinguishes it from the RTO covered in Block 4a.

Analogy: a kitchen timer that only counts while you hold the start button — release before it dings and it resets to zero.

Live demo: wire the TON above so **Q** drives a lamp output, 5-second PT. Open the software's data monitor and watch the **.ACC/ET** value tick upward in real time (usually in the underlying scan-rate resolution — commonly 1ms or 10ms increments depending on platform). Show both a full-duration hold (Q asserts) and an early release (ET snaps back to

0, Q never asserts). Run a third variant: release right at the boundary (e.g., at 4.9s of a 5s preset) so students see explicitly that "almost done" still means "fully reset," reinforcing that there's no partial credit in this instruction's behavior.

Precision/resolution note worth raising here: most PLC timer instructions have a stated resolution (e.g., 1ms) that is separate from scan time. A timer can, in principle, be more precise than the scan cycle it's evaluated within, since the underlying accumulation is driven by a hardware clock; however, the *observable* update of $\textcircled{Q}$ and $\textcircled{ET}$ in the program is still bound to when the next scan's logic-solve phase runs. State plainly: a timer preset of "5000ms" may actually assert $\textcircled{Q}$ on a scan slightly after the 5000ms mark, never before — worth knowing for any application with tight timing tolerances.

1b. TOF (Timer Off-Delay) — 12 min

Function block signature:

```
TOF_Instance(IN := MotorRunning, PT := T#30s);  
Q := TOF_Instance.Q; // true immediately on IN rising edge, stays true PT  
after IN falls
```

Internal state machine:

```
IF IN = TRUE THEN  
    Q := TRUE  
    ET := 0  
ELSE  
    IF ET < PT THEN  
        ET := ET + (time since last scan)  
        Q := TRUE  
    ELSE  
        Q := FALSE  
    END_IF  
END_IF
```

Timing diagram:



Real-world rung — cooling fan example, written out fully:

```

|---[ Motor_Running ]------( TOF )---|
                                Timer: FanRunOn
                                Preset: 30000 (ms)

|---[ FanRunOn.Q ]------( Fan_Output )---|

```

Note explicitly: $\textcircled{Q}$ on a TOF goes true on the *same scan* IN goes true (zero delay on rising edge) — the delay only applies to the falling edge. This asymmetry is exactly what confuses students in 1d.

Second real-world example, to reinforce the pattern with a different domain: a security floodlight triggered by a motion sensor, wired so the light stays on for 2 minutes after the sensor last detected motion, rather than switching off the instant motion stops being detected (which would be visually jarring and defeats the purpose of the light in the first place). Ask the class to identify $\textcircled{IN}$ (motion sensor) and what $\textcircled{PT}$ should be (120,000ms) before revealing the rung, so they're applying the mental model rather than just watching it applied.

1c. Preset Value Experimentation — 6 min

Have students edit $\textcircled{.PRE}$ / $\textcircled{PT}$ live (e.g., change 5000ms to 2000ms) and re-run the demo, watching $\textcircled{.ACC}$ / $\textcircled{ET}$ reach the new value faster. If the platform supports online edits without a full download, use that to keep iteration fast. Have at least one student try an unusually short preset (e.g., 100ms) and observe that $\textcircled{Q}$ still asserts reliably as long as PT exceeds one scan's worth of time — then ask what would happen if PT were set shorter than a single scan time, to seed the idea that scan time imposes a practical lower bound on usable timer resolution (full scan-time discussion was covered Day 1, Block 3 — callback explicitly).

1d. Common Student Mistake — flag proactively

TON vs TOF direction confusion. Put both timing diagrams from 1a/1b on the board simultaneously and have students point to the exact scan-cycle moment each $\textcircled{Q}$ transitions high and low. A second, related mistake worth flagging here: students sometimes assume a TOF's $\textcircled{Q}$ starts false and only goes true after the delay — this is backwards; TOF's delay is on the *falling* edge, meaning $\textcircled{Q}$ is immediately true on $\textcircled{IN}$ going true, and only the *turn-off* is delayed. Say this distinction twice in different words if confusion persists, since the instruction names alone actively mislead a first-timer's intuition.

2. Counters (CTU, CTD) + Edge Detection — 35 min

2a. CTU (Count Up) — 8 min

Function block:

```

CTU_Instance(CU := PartSensor, RESET := ResetBtn, PV := 10);
CV := CTU_Instance.CV;    // INT/DINT, current count
Q  := CTU_Instance.Q;     // BOOL, true when CV >= PV

```

- **(CU)** — count-up input, **rising-edge sensitive by instruction design** (increments once per false→true transition, not while held true)
- **(RESET)** — BOOL, zeroes CV when true
- **(PV)** — INT, preset value (Allen-Bradley: **(.PRE)**)
- **(CV)** — INT (Allen-Bradley: **(.ACC)**), current accumulated count
- Data type note: Allen-Bradley counters typically use DINT (32-bit signed, range -2,147,483,648 to 2,147,483,647); Siemens S7 counters (**(S_CU)**) are often a native 16-bit counter word range 0-999.

Internal behavior, precisely: the counter instruction itself maintains a hidden "previous CU state" bit internally — this is why a raw CTU wired directly to a held-true input does *not* actually re-trigger every single scan on most platforms; the instruction's own internal edge-detection on **(CU)** prevents that. State this carefully, because it sets up an important nuance for Block 2c: the CTU's *own* count input is edge-sensitive by instruction design, but a plain contact-to-coil rung is not — the edge-detection problem in 2c is about signals feeding *into* other logic that lacks this built-in edge sensitivity, or about explicitly wanting a single pulse for purposes beyond just the counter itself (e.g., triggering a log entry once, as seen later on Day 3).

Overflow behavior worth mentioning: most CTU implementations do not wrap silently past their data type's maximum value in normal operation — many platforms simply stop incrementing once **(CV)** reaches the maximum representable value for its type, though this is platform-dependent and worth checking against the lab's specific PLC brand rather than assumed.

2b. CTD (Count Down) — 7 min

```
CTD_Instance(CD := PartRemoved, LOAD := LoadBtn, PV := StartingCount);
CV := CTD_Instance.CV;      // decrements toward 0
Q  := CTD_Instance.Q;      // true when CV <= 0
```

Contrast the reset semantics explicitly: CTU resets CV *to zero*; CTD typically *loads* CV to a starting PV value via a LOAD input rather than "resetting" — different instruction, different parameter name, worth stating so students don't assume symmetry that isn't there.

Combined CTUD note (some platforms offer a single up/down counter instruction): a

(CTUD) combines both behaviors into one instance with separate **(CU)** and **(CD)** inputs sharing one **(CV)** — worth a one-line mention if the lab platform supports it, since it's common enough in real applications (e.g., tracking net inventory where parts both arrive and leave) that students should at least recognize the name if they encounter it later.

2c. Edge-Triggered Instructions — R_TRIG / F_TRIG — 10 min

Instruction behavior at the scan level (this is the technical core of the whole block):

An R_TRIG maintains one bit of internal state — the value of its input from the *previous* scan. Each scan it compares current input to that stored previous value:

```

IF (CurrentInput = TRUE) AND (PreviousInput = FALSE) THEN
    Q := TRUE           // exactly one scan
ELSE
    Q := FALSE
END_IF
PreviousInput := CurrentInput // update for next scan

```

This is why **Q** is true for **exactly one scan** — typically single-digit milliseconds — regardless of how long the physical button stays pressed.

F_TRIG, for completeness, mirrored:

```

IF (CurrentInput = FALSE) AND (PreviousInput = TRUE) THEN
    Q := TRUE
ELSE
    Q := FALSE
END_IF
PreviousInput := CurrentInput

```

Ladder form (Allen-Bradley calls this a One-Shot / OSR — Rising, Siemens uses **R_TRIG FB or a P-edge contact denoted **P** on the rung):**

```

|--[ PartSensor ]--[ONS]----- ( CountPulse )--|

|--[ CountPulse ]----- ( CTU )--|
                               Counter: PartCount
                               Preset: 10

```

Concrete demonstration — run this exactly:

1. Wire logic so a plain contact from **PartSensor** (not a counter, to isolate the concept from CTU's own internal edge-sensing discussed in 2a) directly sets an internal coil **RawPulseCount** via an ADD-and-store rung executed every scan the input is true. Hold the button for ~2 seconds. If scan time is ~10ms, that's roughly 200 scans — the value will increment ~200 times for one physical press (subject to platform scan-time variation — state this caveat explicitly).
2. Insert **ONS**/**R_TRIG** between **PartSensor** and the same increment rung. Repeat the same 2-second hold. The value increments by exactly 1.

Narrate the scan-count math live on the board: "Roughly 10ms scan time, ~2000ms hold ≈ 200 scans — that's why you saw the count jump by roughly 200, not exactly 200, since scan time isn't perfectly fixed." Ask the class to predict, before running it, roughly how many counts a 500ms hold would produce at the same scan rate (~50) — turning it into a quick prediction-then-verify exercise rather than a pure demo.

Why this distinct from CTU's own built-in edge sensing (tie back to 2a explicitly): the CTU instruction already debounces its own `(CU)` input internally. The `R_TRIG/ONS` lesson here is about the *general* pattern — any rung that increments a value, sets a flag, or triggers an action on every scan a condition is true needs explicit edge detection if the intent is "once per event," because plain contact-to-coil logic has no built-in memory of previous scans. This generalizes far beyond counters — it directly resurfaces on Day 3 for alarm/event logging.

2d. Project: Edge-Triggered Part Counting — 10 min

Task: `(PartSensor)` (simulated rising edge) → `(ONS)/(R_TRIG)` → `(CTU)` with `PV=10` → `(Q)` drives `(BatchComplete)` output; separate `(ResetBtn)` → counter `(RESET)`.

While circulating, check specifically: is `(ONS)` placed before the counter's count input, or mistakenly wired to the reset line? Ask: "What happens if you hold the reset button down as currently wired?" to surface the misapplication if present. A second check worth making: does `(ResetBtn)` itself need its own edge detection wired to the `(RESET)` input, or is a held-true reset acceptable? Discuss briefly — for a `RESET` input, holding true generally just holds CV at zero rather than causing repeated miscounting, which is a meaningfully different situation from the `CU` input case, worth having students articulate the distinction themselves.

3. Online Monitoring & Forcing I/O — 25 min

3a. Watching Live Rung States — 8 min

Energized rungs/contacts highlight (commonly green, or bold outline) in online/monitor mode. Have students read a live rung and state the truth value of each contact and the resulting coil state without prompting. Extend this into reading `(.ACC)/(ET)` values directly off timer and counter instructions on-screen (not just their `BOOL(Q)` outputs) — this numeric-value monitoring is what actually gets used in Block 4's debugging exercise, so make sure students practice locating these fields in the software's instruction display, not just the pass/fail highlight color.

3b. Forcing Inputs/Outputs for Testing — 10 min

Mechanism, stated precisely: forcing intercepts the value between the physical I/O module and the logic engine — a forced input substitutes a software-set value for the actual voltage read at the terminal; a forced output substitutes a software-set value for what gets written to the physical terminal, **regardless of what the logic computed.**

```
Normal:  Physical terminal --> Input Image Table --> Logic --> Output Image
Table --> Physical terminal
Forced:  [FORCED VALUE]    --> Input Image Table --> Logic --> [FORCED VALUE]
--> Physical terminal
```

Use cases: testing logic before field devices are wired; isolating whether a fault is in logic vs. physical wiring/sensor (force the input high — if the downstream logic behaves correctly,

the fault is upstream in the physical wiring or sensor, not the program).

A second, less obvious use case worth naming: commissioning a machine in stages — forcing downstream outputs to verify wiring to actuators before the upstream sensors that would normally trigger them are even installed yet. This is common enough on real installation jobs that it's worth a specific mention rather than treating forcing purely as a bug-hunting tool.

Platform-specific behavior worth flagging: on many platforms, forces persist even through a program download or a PLC mode change (RUN to PROGRAM and back), and some platforms display a distinct "FORCE" indicator LED specifically to warn that active forces exist somewhere in the system — precisely because a forgotten force left active from a previous testing session is a genuine, recurring real-world hazard. State plainly: always check for and clear active forces before leaving a system in a production state.

Safety caveat, stated explicitly: a forced output bypasses interlocks entirely because forcing happens *after* the logic solve, overwriting whatever the logic — including any safety interlock rungs — computed. On a live system this requires lockout/tagout procedures; this is a training/simulator-only exercise as presented here.

3c. Reading Fault/Diagnostic Indicators — 7 min

Module fault LEDs (commonly a red/amber "FLT" or "ERR" LED per module), and CPU run/stop/fault state in software (commonly shown as RUN / STOP / FAULT / PROG in the status bar). Frame as the first diagnostic step: hardware/comm fault vs. logic-level fault are different failure domains requiring different next steps.

Expand with a short concrete triage sequence to teach as a habit: (1) check CPU state first — if it's in FAULT, check the fault log/history for a specific fault code before touching logic; (2) check module-level LEDs next — a module fault often points to a specific I/O channel or a communications issue on the backplane; (3) only after ruling out both of the above, move into the online monitoring tools from 3a to inspect logic-level behavior. State explicitly why this order matters: chasing a logic bug for twenty minutes when the actual problem is a faulted I/O module is a very common, very avoidable waste of troubleshooting time.

Project (folded into this block): Forcing Exercise

Force `(PartSensor)` high without physical actuation, confirm `(CTU.CV)` increments. Force `(Fan_Output)` on/off directly, independent of `(FanRunOn.Q)`. Ask students to state explicitly which fault category (hardware vs. logic) each forcing action would help isolate in a real troubleshooting scenario. As a closing check for this block, have students locate and clear any forces they created, reinforcing the "always clear your forces" habit from 3b before moving on to Block 4.

4. Combine Timer + Counter, Group Troubleshooting — 25 min

4a. Retentive vs. Non-Retentive Timers — 10 min

RTO (Retentive Timer On) function block:


```
RTO_Instance(IN := MotorRunSignal, PT := T#8h);  
ET := RTO_Instance.ET; // accumulates across multiple IN pulses, survives  
IN going false  
RES_Instance(RTO_Instance); // separate explicit reset instruction required
```

Key technical distinction: a standard TON's (ET)/(ACC) is implicitly cleared whenever (IN) goes false. An RTO's (ET)/(ACC) is **held** across an (IN) false period and only cleared by an explicit (RES) instruction — meaning without that reset rung, the value persists even through a PLC power cycle if the platform's retentive memory is battery-backed or stored in non-volatile memory (worth flagging as a platform-dependent detail).

Use case, addressed concretely: total motor run-time across multiple start/stop cycles, e.g., preventive-maintenance interval tracking (motor needs service every 8 accumulated running-hours regardless of how many separate times it was started and stopped).

A second concrete use case, to broaden beyond just the motor-runtime example: tracking cumulative machine-cycle time for a shift-based production report — e.g., accumulating total "machine actively producing" time across an 8-hour shift with multiple short stops for material changeovers, where each individual stop should not erase the running shift total. Ask students to identify why a plain TON would be unusable for this (it would reset to zero on every changeover stop, discarding all previously accumulated time) before revealing the RTO as the fix — reinforces the distinction by contrast rather than definition alone.

Retentive counters, briefly: most CTU/CTD instructions are inherently retentive by default (CV holds its value across IN going false, since CU is edge-based rather than level-based) — contrast this explicitly with timers, where retentive behavior is the *exception* (RTO) rather than the default (TON). This asymmetry between timer and counter default behavior is worth stating directly, since it's easy for students to assume the two instruction families behave symmetrically when they don't.

4b. Live Debugging Exercise — 10 min

Pre-plant faults into a working program, e.g.:

1. Remove the (ONS)/(R_TRIG) in front of a (CTU) — count over-increments on a held input.
2. Swap an (RTO) for a (TON) in a cumulative-runtime rung — accumulated value resets unexpectedly every time the motor stops.
3. A third fault worth planting for variety: an (RTO) used where a plain (TON) was actually needed (e.g., a delay-before-alarm rung that should reset cleanly each time the alarm condition clears, but instead keeps accumulating stale elapsed time from a previous unrelated trigger because no (RES) rung exists) — this demonstrates that retentive behavior is not universally "better," it's situation-dependent, and using it where it isn't needed creates its own class of bug.

Have students use Block 3's online monitoring (live rung state, (ACC)/(ET) values, forcing) to localize each fault rather than reading source code top to bottom — this mirrors actual field

troubleshooting workflow. Encourage a specific diagnostic habit: check the instruction's live `(.ACC)`/`(ET)` value first, compare it against what the rung's *intended* behavior would produce at that point in the test sequence, and only then look at the rung's actual wired logic to find the discrepancy — numeric-value-first troubleshooting rather than logic-read-first.

4c. Project: Counter with Built-In Delay — 5 min

Rung structure:

```
|--[ PartSensor ]--[ONS]--( DebouncePulse )--|  
  
|--[ DebouncePulse ]------( TON )--|  
                                Timer: DebounceTimer  
                                Preset: 3000 (ms)  
  
|--[ DebounceTimer.Q ]------( CTU )--|  
                                Counter: PartCount
```

State explicitly: the sensor must produce a *stable* signal for the full 3-second window (this rung as drawn requires the input to remain continuously true, not just pulse — flag this as worth checking against the actual simulated sensor behavior, since a true debounce-before-count pattern sometimes needs the timer gated differently depending on whether the "noise" is a single glitch or a sustained but jittery signal). Note the instruction ordering explicitly: `(ONS)` here produces a single pulse that is *not* what feeds the counter directly — instead it feeds a TON, and the TON's completion (a stable condition sustained the full preset) feeds the counter. This is a meaningfully different composition than Block 2's simple ONS-directly-into-CTU pattern, worth having students articulate the difference in their own words before moving to the Day Project.

5. Day Project — Traffic Light Controller — 95 min

Goal: Red → Green → Yellow → Red, looping, timed by TONs, driven by an explicit state variable.

State variable implementation — two viable technical approaches, mention both:

- **Option A — one INT state register** (`TrafficState`), values 0/1/2 for Red/Green/Yellow), compared with `(EQU)` instructions to gate each branch.
- **Option B — three mutually exclusive BOOL bits** (`State_Red`), (`State_Green`), (`State_Yellow`)), each set by the prior state's timer completion and cleared by its own timer completion.

State explicitly, as a framing point before design begins, why this project is structurally different from anything built so far this week: every previous rung (Days 1 and 2 up to this point) has been purely *combinational* — its output depends only on the current scan's input values. A state machine is *sequential* — its output depends on both current inputs and some

remembered history (which state it's currently in). This distinction — combinational vs. sequential logic — is worth naming explicitly on the board, since it's the conceptual leap this project is actually testing, not just "more timers."

1. Design — 10 min

State diagram to sketch:

```
[Red, TON_Red PT=10s] --Q--> [Green, TON_Green PT=8s] --Q--> [Yellow,
TON_Yellow PT=3s] --Q--> back to Red
```

Have students also annotate, for each state on their sketch, exactly which output(s) are active and which timer is running — this forces an explicit mapping between "state" (an abstract idea) and "concrete PLC tags" (Red_Light, TON_Red, etc.) before any code is written, which is where most of the design confusion for a first state-machine project actually originates.

2. Build — 35 min

Sample rung set (Option A, INT state register, Allen-Bradley style):

```
|--[ TrafficState EQU 0 ]------( Red_Light )--|
|--[ TrafficState EQU 0 ]------( TON_Red )--|
                                           Preset: 10000
|--[ TON_Red.Q ]-----[MOV 1, TrafficState]--( RES TON_Red )--|

|--[ TrafficState EQU 1 ]------( Green_Light )--|
|--[ TrafficState EQU 1 ]------( TON_Green )--|
                                           Preset: 8000
|--[ TON_Green.Q ]-----[MOV 2, TrafficState]--( RES TON_Green )-
-|

|--[ TrafficState EQU 2 ]------( Yellow_Light )--|
|--[ TrafficState EQU 2 ]------( TON_Yellow )--|
                                           Preset: 3000
|--[ TON_Yellow.Q ]-----[MOV 0, TrafficState]--( RES TON_Yellow
)--|
```

Note the explicit **(RES)** (reset) instruction on each timer at the moment of state transition — this is the exact mechanism that prevents the bug flagged in step 3.

Sample rung set (Option B, BOOL-bit state, for contrast — worth building or at least sketching alongside Option A if time allows):

```

--[ State_Red ]----- ( Red_Light )--|
--[ State_Red ]----- ( TON_Red )--|
                                     Preset: 10000
--[ TON_Red.Q ]--[SET State_Green]--[RES State_Red]--( RES TON_Red )--|

--[ State_Green ]----- ( Green_Light )--|
--[ State_Green ]----- ( TON_Green )--|
                                     Preset: 8000
--[ TON_Green.Q ]--[SET State_Yellow]--[RES State_Green]--( RES TON_Green )--|

--[ State_Yellow ]----- ( Yellow_Light )--|
--[ State_Yellow ]----- ( TON_Yellow )--|
                                     Preset: 3000
--[ TON_Yellow.Q ]--[SET State_Red]--[RES State_Yellow]--( RES TON_Yellow )--|
-|

```

Note here the explicit **(SET)**/**(RES)** pair used on the state bits themselves at each transition — this guarantees mutual exclusivity only if implemented correctly on every transition; a missed **(RES)** on the outgoing state bit is a distinct, additional bug class beyond the timer-reset bug, worth mentioning if Option B is actually built by any student group.

3. Test & Debug — 35 min

Root-cause of the most common bug, explained at instruction level: if the **(RES TON_Red)** instruction is omitted, **(TON_Red.ET)/(.ACC)** is never explicitly zeroed. Depending on platform behavior, a TON whose **(IN)** condition goes false (because **(TrafficState)** no longer equals 0) will typically auto-reset its own **(ET)** to 0 the next scan anyway — **but** if the same timer instance is reused or the state comparison logic has any overlap/race condition across the transition scan, stale **(ET)** values can cause a short first pulse on the next cycle through the loop. Have students verify, using the **(.ACC)/(.ET)** monitor from Block 3a, that each timer's accumulated value is genuinely 0 at the exact scan its state becomes active again.

A second bug class worth watching for, specific to Option B builds: forgetting the **(RES)** on the outgoing state bit results in two state bits being simultaneously true for one or more scans, meaning two lights could briefly both be active — ask students building Option B to specifically verify, using live monitoring, that at every point in the running cycle exactly one state bit is true, never zero and never two.

A third, more subtle timing bug worth raising if a group finishes early on debugging: if the transition rung and the timer-gating rung are evaluated in an order where the state changes mid-scan-solve before the old state's timer rung has been evaluated that same scan, some platforms' scan order (top-to-bottom, single pass) can produce a one-scan glitch depending on rung placement. This is a genuinely deeper rabbit hole — mention it only as a "for the curious" aside, not a required debugging target for this project.

4. Class Walkthrough — 15 min

Compare Option A (INT state + EQU) vs. Option B (three BOOL bits) implementations submitted by different students. Discuss trade-off explicitly: INT state scales better to more states (a 10-state sequencer needs one register either way) but requires an EQU comparison per state; BOOL-bit state is more readable at 3 states but doesn't scale cleanly past a handful, and introduces the extra `SET`/`RES` mutual-exclusivity bookkeeping discussed above that INT/EQU avoids entirely by design (only one value can ever equal a given comparison target at once, with no separate exclusivity logic required).

If finished early: Add a `PedestrianBtn` input that, when true, forces `TrafficState` to a new interrupt state (e.g., state 3 = all-red + walk-signal output) regardless of the current normal-cycle state, then returns to state 0 (Red) after its own TON expires. This requires an explicit priority check at the top of the rung logic — e.g., testing `PedestrianBtn` *before* the normal state-transition rungs — a direct preview of priority/override logic needed in more complex sequencing later. Ask students to consider explicitly: should a pedestrian request interrupt an in-progress Green phase immediately, or should it be "latched" and only take effect once the current phase naturally completes? Both are defensible design choices with different real-world tradeoffs (immediate safety response vs. avoiding an abrupt, potentially confusing light change to approaching traffic) — there's no single correct answer, and surfacing that ambiguity is itself a useful discussion for a first exposure to interrupt/override design.